

dr. Matej Šprogar

Analiza algoritmov

ocenjevanje zahtevnosti algoritmov



Kaj je analiza algoritmov?

Berljivost, pravilnost, varnost itd. **žal niso** enostavno določljive lastnosti programov, zato pa lahko analiziramo vsaj njihovo obnašanje...

Analiza algoritma je **teoretična** študija, ki pregleda in ovrednoti njegovo obnašanje (*performance*) in učinkovitost (porabo virov).

Zanimajo nas odgovori na naslednja vprašanja:

- kateri algoritem je najhitrejši oziroma kako raste časovna zahtevnost algoritma s količino podatkov?
- kateri algoritem porabi najmanj pomnilnika oziroma kako raste prostorska zahtevnost algoritma?

Zakaj in kako analiziramo algoritme?

Določeni realni problemi so računsko izredno zahtevni. Zanima nas, kateri od ustreznih algoritmov je boljši v podanem primeru; mogoče je dovolj nakup novega(ih) procesorja(ev) ali pa problem sploh ni rešljiv z računalnikom...

Za osnovno primerjavo bi lahko algoritme sprogramirali ter izmerili in primerjali realne čase izvajanja. Žal so takšni rezultati ponavadi pristranski (odvisni od kvalitete programov, prevajalnikov, podatkov...), zato zaradi *primerljivosti* raje analitično izračunamo število korakov ali operacij, ki jih mora algoritem izvesti.

Analiza zahtevnosti

T(n) - časovna zahtevnost je število akcij/korakov/operacij/ukazov, ki jih algoritem izvede v odvisnosti od obsega (števila) podatkov. Merimo lahko vsako dejansko izvedeno operacijo ali (enostavneje) le ključne ukaze algoritma.

S(n) - prostorska zahtevnost je potrebna količina pomnilnika za izvedbo algoritma v odvisnosti od obsega podatkov. Izrazimo jo v odvisnosti od vhodne količine podatkov.

Obe zahtevnosti sta pogosto* *obratno sorazmerni*: hitrejši kot je algoritem, več prostora rabi, z manj prostora pa zmore le počasnejši algoritem.

Primer

Štetje korakov je vedno subjektivno - v primeru na desni bi lahko **osrednji** ukaz šteli kot *eno* operacijo, čeprav opravi indeksacijo, seštevanje in pisanje spremenljivke (torej 3 operacije); dalje, seštevanje pomeni tudi branje... In različni ukazi trajajo različno dolgo, različni jeziki imajo različno število ukazov...

Na srečo se izkaže, da je dovolj, če štejemo le **najpomembnejše** operacije, ki se izvedejo največkrat. V tem primeru seštevanje skupne vsote.

```
int i = 0;           // 1
while (i < N) {       // 1
    sum += polje[i];  // 1
    i += 2;           // 1
}
```

$$T(n) = 1 + n/2 * (1 + 1 + 1) + 1 = 3n/2 + 2$$

$$T(n) = 1 + n/2 * (1) = n/2 + 1$$

Ocena zahtevnosti $O(n)$

Razlike v hitrosti/število posameznih korakov (operacij) lahko namreč vplivajo kvečjemu na konstante pri posameznih členih izraza - lahko bi dobili recimo $1.7n + 2.5$ namesto $1.5n + 1$...

Kar ne spremeni dejstva, da se mora izvesti vsaj n osnovnih operacij.

Zato nas bolj kot dejanska formula časovne ali prostorske zahtevnosti zanima, kako zahtevnost narašča, ko se n povečuje preko vseh meja...

$O(g(n)) = \{ f(n): \text{kjer } \exists \text{ pozitivni } c_1, c_2 \text{ in } n_0 \text{ tako, da } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ za } \forall n \geq n_0 \}$

$g(n)$ enostavno dobimo tako, da iz $f(n)$ ohranimo samo **najmočnejši** člen **brez** vodilne konstante...

$$T(n) = 5 = O(1)$$

$$T(n) = 6n - 6 = O(n)$$

$$T(n) = 2n^2 + 4n - 6 = O(n^2)$$

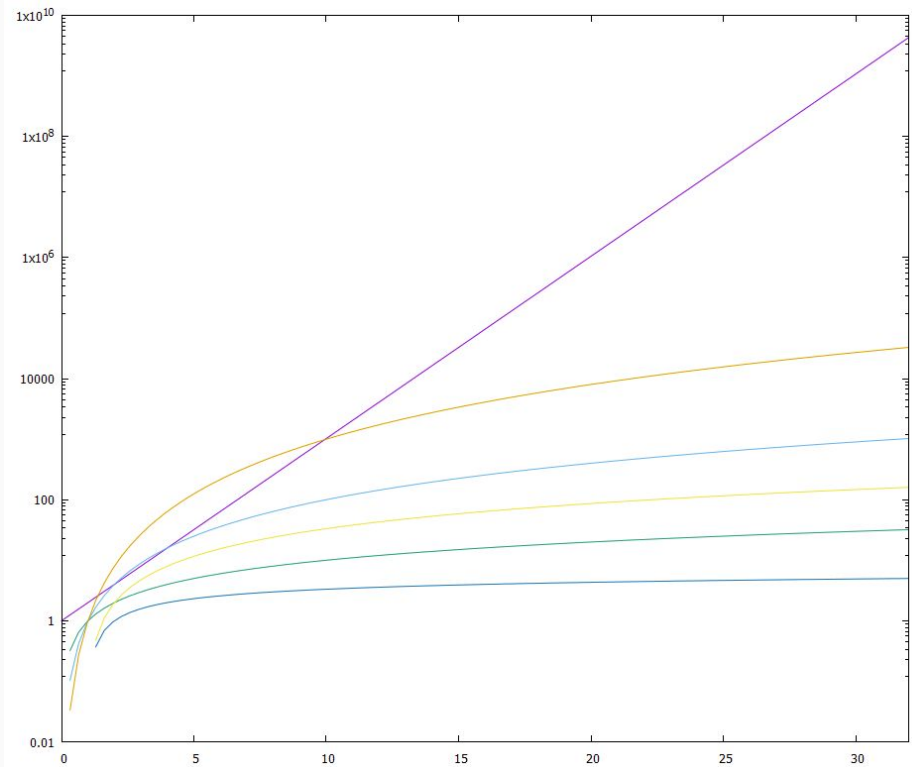
$$T(n) = n^2 + 5n - 6 = O(n^2)$$

$$T(n) = 8 \cdot 2^n + 3n^3 + 4 = O(2^n)$$

Katera zahtevnost je najboljša?

Pri malih n še obstaja določena konkurenca, potem pa je 2^n vedno slabša in predstavlja prevelik problem za trenutne računalnike; algoritmi z $n \cdot \log(n)$ zahtevnostjo ali nižjo so še sprejemljivi, kvadratne ali višje zahtevnosti vzamejo preveč časa...

Recimo, da 1 operacija traja $1 \mu\text{s}$. Algoritem reda $O(n^2)$ bi za $n=10.000$ podatkov potreboval $10^8 \mu\text{s} = 100 \text{ s}$, algoritem reda $O(\log_2(n))$ pa le $13 \mu\text{s}$!



2^n

n^3

n^2

$n \log_2(n)$

n

$\log_2(n)$

Odvisnost od podatkov

Pogosto je ocena zahtevnosti odvisna ne samo od števila, ampak tudi od vsebine podatkov.

Takrat nas večinoma zanimata ocena v *najslabšem* primeru in *povprečna* ocena zahtevnosti; *najboljši* primer bo češnja na torti...

Recimo pri sortiranju so lahko podatki že urejeni (najboljši primer), naključni (povprečni primer) ali urejeni nasprotno (najslabši primer).

```
sort({1,2,3,4,5}); // najboljši primer  
sort({5,4,3,2,1}); // najslabši primer  
sort({2,4,1,5,3}); // povprečen primer
```

Če vemo, da bodo podatki recimo že skoraj (obratno) sortirani, je mogoče smiselno uporabiti kakšen v splošnem slabši algoritem, ki pa deluje v tem specifičnem primeru hitreje, kot če bi uporabili sicer v splošnem hitrejši algoritem. In zato moramo poznati različne algoritme...